

# Committed Composite Approaches

Commitments, blinding, and the partition-leak fix

This note summarizes committed-sort and committed-product. Both keep the same composite architecture as the plain variants:  $K$  independent segment proofs plus one glue proof. The change is the public/private split. Plain variants publish the stitching values needed to connect the proofs, while committed variants publish only blinded commitments to those values.

**Notation.**  $N$  is the number of nodes,  $K$  the number of segments,  $M = \frac{N}{K}$ , root is the Poseidon2 Merkle root of the  $N \times N$  cost matrix, and  $C_i$  is segment  $i$ 's public commitment.

## 1. Problem: the Plain Surface Leaks

The flat proof exposes only  $\{\text{root}, \text{threshold}\}$ . Composite proofs need additional shared values so that the segment proofs and glue proof refer to the same tour decomposition.

- plain-sort publishes each segment's sorted node set, endpoints, and partial cost. The glue sorts the concatenated public node lists and checks  $[0..N-1]$ . This is exact, but the partition is directly disclosed.
- plain-product replaces each public node list with  $P_i = \prod_{v \in S_i} (X + v)$  and publishes hash-chain anchors. This is smaller, but the values are still confirmation oracles: on small domains, or given likely candidates, an observer can recompute public fingerprints and test guesses.

This is not a zero-knowledge failure. ZK hides witnesses, not public inputs. The fix is to move the stitching values into the witness while keeping a short public handle for cross-checking.

## 2. Mechanism: Blinded Commitments

Each segment publishes one commitment

$$C_i = \text{fold}(r_i, \text{values}_i),$$

where the implementation is an iterated Poseidon2 fold:

$$\text{acc}_0 = r_i, \quad \text{acc}_{t+1} = H(\text{acc}_t, v_t), \quad C_i = \text{acc}_\ell.$$

The blinding scalar  $r_i$  is private. The segment circuit proves that its hidden local values open to  $C_i$ . The glue circuit receives the same values and  $r_i$  as private witness, recomputes the same  $C_i$ , and then runs the global checks on the hidden values. The verifier only checks that both proofs expose the same opaque commitment:

```
# committed-product verifier; committed-sort omits X
for shared in ("root", "X"):
    for i, s in enumerate(subs):
        assert s[shared] == glue[shared]
for i, s in enumerate(subs):
    assert glue["C_is"][i] == s["C_i"]
```

The blinding factor is what closes the small-domain brute-force leak. Without  $r_i$ , a commitment would be a deterministic dictionary target: guess a segment, recompute, compare. With private  $r_i$ , a guessed segment cannot be confirmed without also finding an opening value. The hiding is therefore computational, under Poseidon2 random-oracle / preimage-resistance assumptions.

## 3. Noir Essentials

committed-sort folds the private segment nodes and computed partial cost into  $C_i$ :

```
let partial_cost: u64 = sum;
let mut acc: Field = r;
for j in 0..M {
    acc = Poseidon2::hash([acc, cycle_segment[j] as Field], 2);
}
acc = Poseidon2::hash([acc, partial_cost as Field], 2);
assert(acc == C_i, "commitment mismatch");
```

The glue opens the same commitments and runs the exact sort check on private nodes:

```
for i in 0..K {
    let mut acc: Field = r_is[i];
    for j in 0..M {
        acc = Poseidon2::hash([acc, all_nodes[i * M + j] as Field], 2);
    }
}
```

```

    acc = Poseidon2::hash([acc, partial_costs[i] as Field], 2);
    assert(acc == C_is[i], "commitment mismatch");
}
let sorted = all_nodes.sort_via(|a: u32, b: u32| a <= b);
for i in 0..N { assert(sorted[i] == i as u32); }

```

committed-product hides the product fingerprint, chain anchors, endpoints, and cost:

```

let mut prod: Field = 1;
for j in 0..M { prod = prod * (X + cycle_segment[j] as Field); }
let p_i: Field = prod;

let mut acc: Field = r;
acc = Poseidon2::hash([acc, p_i], 2);
acc = Poseidon2::hash([acc, h_in_i], 2);
acc = Poseidon2::hash([acc, h_out], 2);
acc = Poseidon2::hash([acc, start_node as Field], 2);
acc = Poseidon2::hash([acc, end_node as Field], 2);
acc = Poseidon2::hash([acc, partial_cost as Field], 2);
assert(acc == C_i, "commitment mismatch");

```

The product glue opens those values privately, checks the chain, derives  $X = H(c)$ , and enforces

$$\prod_i P_i = \prod_{j=0}^{N-1} (X + j).$$

So committed-product keeps the compact grand-product check but removes the public per-segment oracle. Its partition soundness remains probabilistic: Schwartz-Zippel at the Fiat-Shamir challenge plus the random-oracle assumption for deriving  $X$ .

## 4. Security and Tradeoffs

**Binding.** The same public  $C_i$  must be opened by the segment and by the glue. Opening one commitment to two different value sets would require breaking Poseidon2 collision or second-preimage resistance. This blocks the attack where a segment proof proves one value set and the glue uses another.

**Hiding.** The secret  $r_i$  prevents brute-force confirmation of small-domain guesses. The hiding is computational, not information-theoretic, and depends on Poseidon2 plus enough blinding entropy.

**Correctness.** Commitments only hide and bind the stitch values. They do not replace the TSP checks. The global partition is still enforced by exact sort in committed-sort and by the grand-product check in committed-product; edge costs are still bound to the Merkle root.

**Residual leakage.** The committed variants still reveal  $K$  through the number of segment proofs and commitments. committed-product also keeps public  $X$ . Flat and recursive variants have the cleaner  $\{\text{root}, \text{threshold}\}$  public surface, but recursion pays in-circuit verifier cost.

**Cost.** committed-sort and committed-product reach the same partition-privacy class. Their main difference is cost: committed-sort still sorts  $N$  private nodes and commits to all node values, while committed-product compresses each segment to a few field values before committing.

## 5. Summary Table

| Variant           | Public stitch values            | Partition check                 | Privacy status                                                          |
|-------------------|---------------------------------|---------------------------------|-------------------------------------------------------------------------|
| plain-sort        | sorted_nodes, endpoints, costs  | Exact sort in glue              | Partition disclosed                                                     |
| plain-product     | P_i, anchors, endpoints, costs  | Grand product + FS              | Search-based hiding only; small-domain oracle                           |
| committed-sort    | C_i commitments                 | Exact sort over witness         | Computational hiding via secret $r_i$                                   |
| committed-product | C_i commitments plus public $X$ | Grand product + FS over witness | Computational hiding via secret $r_i$ ; product soundness probabilistic |

**Conclusion.** The committed variants do not change the TSP checks. They change the public surface: the real stitching values become witness data, and the public cross-check handle becomes a blinded Poseidon2 commitment.